AI 활용 가이드

올바른 활용 목적

허용 범위와 규칙

실전 프롬프트 & 검증법

AI는 코딩 기계가 아닌 '지능형 튜터'입니다

지능형 튜터링

개인별 학습 속도에 맞춘
1:1 맞춤형 질의응답

수업 중 발생한
학습 공백 보완

개념 시각화 및 구체화

추상적 프로그래밍 언어
이론(메모리 모델, 스코프 등)
의 직관적 이해

구체적 예시 및
코드 변환을 통한 딥러닝

비판적 사고 및 검증

AI 생성 결과물 속 잠재적
오류(Hallucination) 탐지

이론적 근거를 바탕으로
코드 동작 원리 파악

학문적 성장을 위한 AI 활용 경계선

허용 및 권장 범위 (Do)

이론 학습 도우미
난해한 개념('Call by Name' 등작 방식 등) 추가 설명 요청

코드 리팩토링 및 분석
본인 작성 코드의 효율성 분석 및 개선 방향 제안

디버깅 가이드
에러(Segmentation Fault 등) 발생 원인의 이론적 배경 학습

비교 학습
등일 알고리즘의 함수형 vs 명령형 언어 구현 차이점 비교

금지 및 제한 사항 (Don't)

무단 복사 및 붙여넣기
코드를 이해하지 못한 채 제출하는 행위 절대 금지

학습 목표 위배
실습 목적이 '직접 구현'으로 명시된 과제에서의 AI 코드 생성

과제 제출 시 반드시 지켜야 할 핵심 의무

code

```
/* [AI 활용 여부] ChatGPT를 활용하여 메모리 할당 구조를 학습함 */
Node* newNode = (Node*)malloc(sizeof(Node));
// [본인 설명]: malloc을 통해 Heap 영역에 동적으로 메모리를 할당하며...
```

사용 여부 투명성

모든 과제 제출 시
AI 활용 여부 명시

이해의 증명 (설명 의무)

AI가 생성한 코드 포함 시,
반드시 각 라인의 동작 원리를
본인의 언어로 주석 처리할 것

이해하지 못하고 설명할 수 없는 코드는 본인의 코드가 아님 !

질문의 수준이 결과의 수준을 결정합니다

'Why'를 묻는 프롬프트 엔지니어링: 단순 지시가 아닌 'PL(프로그래밍 언어론) 전문가'처럼 질문하기

👎 Bad Prompt: 단순 코드 요구

C 언어로 연결 리스트 짜줘.

(결과: 학습 효과가 없는 단순 복붙용 코드 생성)

👍 Good Prompt: 용어 활용 및 시각화 요구

C 언어의 **동적 메모리 할당(malloc)**을 사용하여 연결 리스트를 구현하고, 각 노드가 **힙(Heap) 영역**에 어떻게 배치되는지 **메모리 지도**를 그려서 설명해줘.

(결과: PL 이론이 결합된 깊이 있는 맞춤형 해설 생성)

AI의 답변을 의심하라: 3단계 코드 검증 프로세스

⚠ AI는 확률적 모델이므로 틀린 정보(Hallucination)를 제공할 수 있음.

1. Generate (생성)

AI에게 코드 작성 및 관련 개념 설명 요청

→

2. Analyze (분석)

수업에서 배운 핵심 기준을 바탕으로 코드 논리 검토

검토 기준:
타입 시스템, 스코프, 메모리 모델

→

3. Cross-check (교차 검증)

- 리눅스(Linux) 환경에서 직접 컴파일 및 실행
- 메모리 분석 도구(Valgrind 등) 및 Visual Studio Debugger를 활용한 최종 디버깅

성공적인 AI 활용을 위한 3대 황금률

Rule 1. [시각화 도구]

생각의 외주화가 아닌, 어려운 개념의 시각화 도구로 활용할 것

Rule 2. [주석 작성]

단순 복붙은 거절, 모든 코드에 이론적 근거를 바탕으로 주석 달기

Rule 3. [직접 검증]

AI의 답을 맹신하지 말고, 배운 지식과 도구(Valgrind 등)로 직접 교차 검증하기